Chapter 6: Building a Future-Proof Architecture

When you start to build real-world enterprise applications, creating a beautiful UI and delivering a good user experience are important, but not sufficient to create the fundamentals for a successful project. Enterprise applications are usually developed over a span of multiple years by teams made of multiple developers; as the application grows and becomes successful, you will need to onboard new developers into the team to keep up with the pace of business expansion.

For all these reasons, it's critically important to build applications that can easily be tested in an automated way so that as the application grows you can rest assured that none of the changes you're going to introduce will break other features and can be easily maintained. This will mean that when a problem does occur, you won't need 3 days just to understand where the issue might be coming from; this you can move forward in your application adding new features without rewriting it from scratch; and that, when you onboard new developers into the team, they won't need 6 months just to understand how the architecture works.

Over time, many architectural patterns have emerged in the developer ecosystem to help teams to create projects to be sustainable over long periods of time by helping developers to separate the various concerns of the application, including the business logic, UI, and user interaction. In the Windows ecosystem, the **Model-View-ViewModel** (**MVVM**) pattern is one of the most popular ways to achieve this goal, especially if you're building your applications using XAML-based platforms such

as WPF, WinUI, and Xamarin Forms. MVVM, in fact, leverages many of the building blocks of XAML (such as binding) to provide a better way for developers to build complex applications.

In this chapter, we'll explore how to implement the MVVM pattern through the following topics:

- Learning the basic concepts of the MVVM pattern
- Exploring frameworks and libraries
- Supporting actions with commands
- Making your application easier to evolve with dependency injection
- Exchanging information between different classes
- Managing navigation with the MVVM pattern

By adopting all these principles, we'll achieve the goal of building future-proof applications that can handle change (be it in the customer's requirements, the technical implementation, or the adoption of new technology) in a smart way. As you can see, we have a lot of content to cover, so let's get started!

Technical requirements

The code for the chapter can be found here:

https://github.com/PacktPublishing/Modernizing-Your-Windows-Applications-with-the-Windows-Apps-SDK-and-WinUI/tree/main/Chapter06

Learning the basic concepts of the MVVM pattern

Before starting to dig into the MVVM pattern, let's try to better understand the problem we're trying to solve. Let's assume we're building an e-commerce application that enables users to add products to a cart and submit an order. The application will have a summary page with a recap of the order and a button to submit the order. This is how the event handler connected to the **`Button`** control might look like:

```
private void OnBtnOrderClick(object sender, Routed
  EventArgs e)
{
    var itemsInBasket = _basketListBox.Items.Cast
      <IOrderItem>().ToArray();
    if (itemsInBasket.Length <= 0)
    {
        return;
    }
    // Validate Shopping Basket
    if (itemsInBasket.Any(item => item.Quantity < 0 ||
      item.Price <= (decimal)0.0))
    {
        throw new Exception("Shopping card manipulation
          detected");
    }
    // Get discount and calculate total
    var discount = _customerDao.Load(_currentCustomer)
      .Discount;
    var total = itemsInBasket.Sum(item => item.Price *
      item.Quantity * discount);
    MessageBox.Show($"Are you sure you want to order?
Your
      total is {total}.", "Confirmation",
        MessageBoxButton.YesNo,
MessageBoxImage.Question);
    _orderSystem.PlaceOrder(itemsInBasket);
    _basketListBox.Items.Clear();
```

```
}
```

Of course, this code is a simplification of what would happen in a real application, but it's a good starting point to highlight its problems. The main one is that this code is a mix of tasks that belong to different domains:

- We're interacting with the UI layer by accessing the **`Items`** property of the **`_basketListBox`** control or by calling **`MessageBox.Show()`** to display an alert.
- We are declaring some business logic that determines whether the cart is in a valid state before moving on to the purchase process.
- We're interacting with the data layer by calling the **`_orderSystem.PlaceOrder()`** method, which will likely store the order in a database.

This approach makes it really complex to do the following:

- **Test the code**: Testing is difficult since all the domains are interconnected. We can't test whether the business logic to validate the cart is correct without raising the event handler or interacting with the database.
- **Maintain the code**: Let's say that the order submission fails. Where is the problem? In the business logic? In the data layer? In the UI layer?
- **Evolve the code**: Let's assume we have a new requirement, and we have to adjust the way discounts are applied. The task will be executed by a developer who just joined the team and is still ramping up on the project. How can he quickly identify where the change must be made if the code that calculates the discount is mixed with other domains?

Let's read into this in more detail.

Moving a bit deeper into the MVVM pattern

Here, we will go into further detail about how the MVVM pattern is split and how it is implemented.

The goal of the MVVM pattern is to ideally split the application into three different layers:

- **The Model**: This domain includes all the entities and services that work with the application's data. For example, in the previous example of the e-commerce application, the Model would contain the `Order` and the `Product` classes and a data service that reads and writes data to the database. The model should be completely agnostic to the UI platform. You should be able to take your model layer and reuse it without changes in any other project based on the .NET ecosystem, such as a Blazor website in Blazor or a mobile application built with MAUI.
- **The View**: This is the UI layer, which contains the definitions of pages, animations, and so on. This is the only layer that is specific to a given platform. In the case of WinUI applications, it's made by all the XAML files and their corresponding code-behind classes.
- **The ViewModel**: This is the glue between the Model and the View. The model retrieves the data to be displayed in the UI but, first, it's processed by the ViewModel so that it can be presented in the right way. On the other side, when the user interacts with the UI, the data gets collected and processed by the ViewModel before sending it back to the model. In a WinUI application, most of the tasks that are performed in a normal application by the code-behind class (such as managing the user interaction) are now moved to the ViewModel layer. The key difference is that code-behind classes have a tight relationship with the UI: you can't, for example, store your code-behind class in a different project than the one containing the XAML files. A ViewModel, instead, is considered a **Plain Old CLR Object** (**POCO**): it doesn't have any specific dependency and, as such, it can be easily stored in a class library, tested, and so on.

The MVVM pattern has become popular in the XAML ecosystem since it relies heavily on the features of the XAML language, as you can understand from the following diagram:

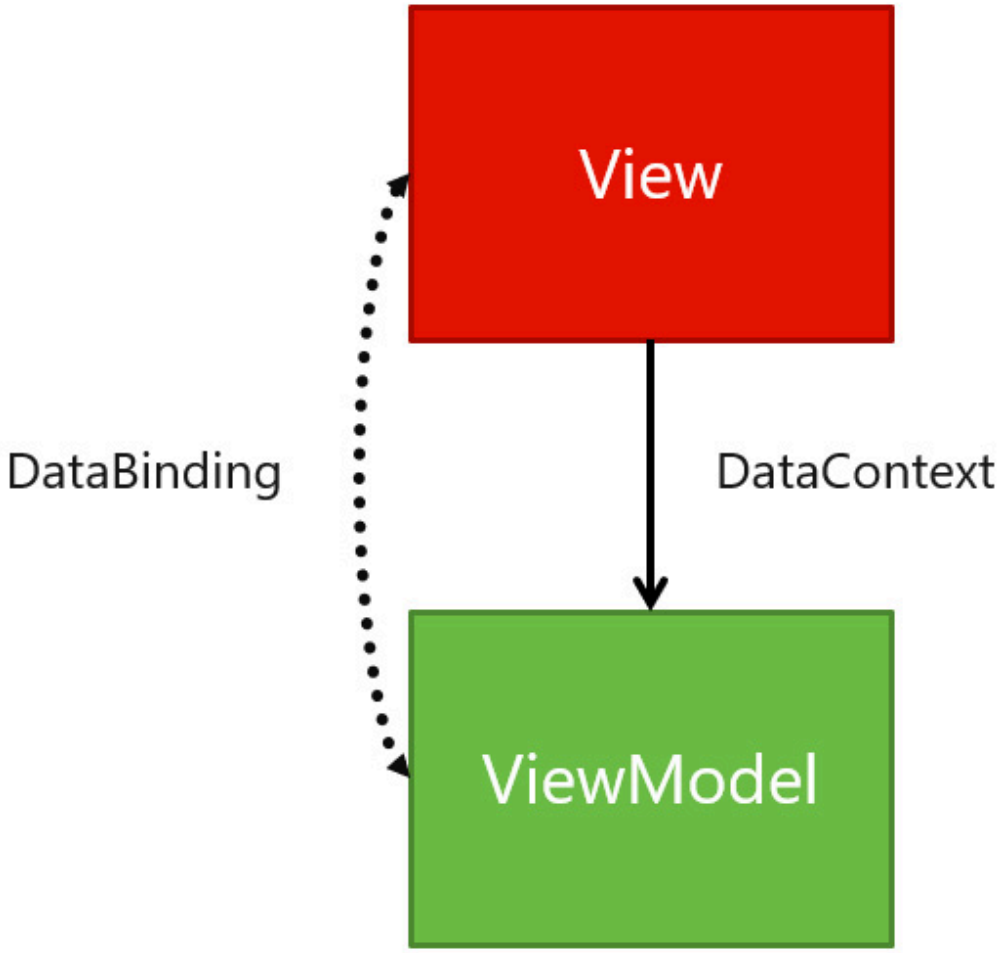

Figure 6.1 – The connection between View and ViewModel is established thanks to binding

In a traditional application, you can reference the controls in the UI simply by setting up a name using the **`x:Name`** property. When you switch to the MVVM pattern, instead, you don't have this direct connection anymore, so you replace it using data binding. Every page or component in your application is backed by a ViewModel class, which is set as **`DataContext`** of the whole page. This enables your XAML pages to access to all the properties exposed by the ViewModel so that you can display the data coming from the Model layer.

Let's understand this concept better with a concrete example. Let's assume that you're building a form to collect the name and surname of a user. In a traditional application, the XAML would look as follows:

```
<StackPanel>
    <TextBlock Text="Name" />
    <TextBox x:Name="txtName" />
    <TextBlock Text="Surname" />
    <TextBox x:Name="txtSurname" />
    <Button Content="Add" Click="OnAddPerson" />
</StackPanel>
```

Since in the code-behind we have direct access to the UI elements, we can directly access the **`Text`** property of the **`TextBox`** controls to retrieve the name and surname inserted by the user:

```
private void OnAddPerson(object sender,
  Microsoft.UI.Xaml.RoutedEventArgs e)
{
    string name = txtFirstname.Text;
    string surname = txtLastname.Text;
    //save the user in the database
}
```

This approach, however, creates a tight connection between the UI and the business logic. We can't move this logic to another class, it must stay in the code behind. Let's translate it into a ViewModel. We have mentioned how a ViewModel is just a simple class, so right-click on your project, go to **Add** | **New class**, and give it a meaningful name (such as **`MainViewModel`**). Since the View can access the ViewModel data through properties, we need first to expose the two bits of information we need to collect (name and surname) in the ViewModel:

```
public class MainViewModel
{
    public string Name { get; set; }
    public string Surname { get; set; }
```

```
}
```

Before using these properties in the UI, we need first to connect the ViewModel class to the View. In other XAML technologies such as WPF, the way to do this was by creating a new instance of the ViewModel and setting it as **`DataContext`** of the page, like in the following sample:

```
public partial class MainView : Page
{
    public MainView()
    {
        InitializeComponent();
        this.DataContext = new MainViewModel();
    }
}
```

As we learned in ***Chapter 3***, *The Windows App SDK for a WPF Developer*, however, WinUI supports a more powerful binding expression called **`x:Bind`** which, instead of being evaluated at runtime, is compiled together with the application. This makes binding faster, more efficient, and type safe as you can find binding errors before running the application.

One of the key differences between traditional binding and **`x:Bind`** is that, if the former uses the **`DataContext`** property exposed by every XAML control to resolve the binding expression, the latter looks for properties declared in code-behind. As such, the most common approach adopted in WinUI applications to connect a View with the ViewModel is to expose the ViewModel as a property in the code-behind, like in the following sample:

```
public sealed partial class MainPage : Page
{
    public MainViewModel ViewModel { get; set; }
    public MainPage()
    {
        this.InitializeComponent();
```

```
            ViewModel = new MainViewModel();
        }
    }
```

Now you can use the **`ViewModel`** property together with the **`x:Bind`** markup expression to connect these properties to the UI:

```
    <StackPanel>
        <TextBlock Text="Name" />
        <TextBox Text="{x:Bind ViewModel.Name,
    Mode=TwoWay}" />
        <TextBlock Text="Surname" />
        <TextBox Text="{x:Bind ViewModel.Surname,
          Mode=TwoWay}" />
        <Button Content="Add" Click="OnAddPerson" />
    </StackPanel>
```

We no longer need to set the **`x:Name`** property to get access to the name and surname entered by the user. Thanks to binding, we can retrieve this information directly through the **`Name`** and **`Surname`** properties of our ViewModel. Since we have set the **`Mode`** property to **`TwoWay`**, it means that not only will the UI be updated when the two properties change, but also that we'll be able to access the values entered by the user in the ViewModel.

However, if you remember what we learned about binding in ***Chapter 2***, *The Windows App SDK for a Windows Forms Developer*, you should realize that the example we have created so far has a flaw: **`Name`** and **`Surname`** in the **`MainViewModel`** class are two simple properties; whenever one of them changes, the UI isn't notified and, as such, the controls won't update. **`MainViewModel`** is a simple class, which means that the way to solve this problem is by implementing the **`INotifyPropertyChanged`** interface, which gives us a way to dispatch notifications to the UI layer when one of the properties change.

This is how a proper implementation of the **`MainViewModel`** class looks:

```
public class MainViewModel: INotifyPropertyChanged
{
    private string _name;
    public string Name
    {
        get { return _name; }
        set
        {
            NotifyPropertyChanged();
            _name = value;
        }
    }
    private string _surname;
    public string Surname
    {
        get { return _surname; }
        set
        {
            NotifyPropertyChanged();
            _surname = value;
        }
    }
    public event PropertyChangedEventHandler
      PropertyChanged;
    private void
NotifyPropertyChanged([CallerMemberName]
      string propertyName = "")
    {
        PropertyChanged?.Invoke(this, new
          PropertyChangedEventArgs(propertyName));
    }
}
```

Our ViewModel now inherits from the **`INotifyPropertyChanged`** interface, which requires us to implement the **`PropertyChanged`** event. To

make the implementation easier to use, we define a method called **`NotifyPropertyChanged()`**, which will invoke the event for us, dispatching the notification to the UI layer. Then, as the last step, we must change the implementation of the **`Name`** and **`Surname`** properties. Instead of using the standard initializer, we have to add some logic to the setter: when the property changes, other than storing its value, we must call **`NotifyPropertyChanged()`** to dispatch the notification to the binding channel.

This code works perfectly fine, but there's space for improvement. What if our application has 10-15 ViewModels? We would need to repeat the **`PropertyChanged`** implementation for each of them. This is one of the areas where libraries can help you in speeding up the implementation of the MVVM pattern.

Exploring frameworks and libraries

MVVM is an architectural pattern, not a library or a framework. However, when you start adopting the MVVM pattern in a real project, you need to have a series of building blocks that can help you speed up the implementation, by avoiding the need to reinvent the wheel at each step. In the previous section, we have seen one of these scenarios: each ViewModel class must implement the **`INotifyPropertyChanged`** interface, otherwise, the connection between the View and the ViewModel would be broken.

This is where libraries and frameworks become helpful: they give you a set of ready-to-use building blocks that you can use to quickly start building all the components of your application. The library we're going to leverage in the examples of this chapter is called the **MVVM Toolkit** and is a part of the Windows Community Toolkit from Microsoft. It's the spiritual successor of the MVVM Light Toolkit by

Laurent Bugnion, one of the most popular MVVM libraries used in the past decade to support the development of XAML-based technologies, since it follows the same principles. The MVVM Toolkit is a very light-weight and simple framework. This means that you might need to do some extra work if you need to manage more complex scenarios; however, thanks to its flexibility, it's very easy to extend and learn. The MVVM Toolkit also makes it easy to onboard new developers into the team: since all its features are based on basic XAML concepts, it's very easy to learn it even if you have never worked with it before.

The first step to use the MVVM Toolkit is to right-click on your project, click on **Manage NuGet packages**, and look for the package with **CommunityToolkit.Mvvm** as an identifier. Once you have installed it, let's see how we can refactor our **`MainViewModel`** class:

```
public class MainViewModel : ObservableObject
{
    private string _name;
    public string Name
    {
        get { return _name; }
        set { SetProperty(ref _name, value); }
    }
    private string _surname;
    public string Surname
    {
        get { return _surname; }
        set { SetProperty(ref _surname, value); }
    }
}
```

Inside the **`CommunityToolkit.Mvvm.ComponentModel`** namespace, we find a class called **`ObservableObject`** that we can use as the base class for our ViewModel. Thanks to this class, we don't need to manually implement the **`PropertyChanged`** event anymore. We can simply use the **`SetProperty()`** method in the setter of our property, which takes care

of storing the value and dispatching the notification to the binding channel.

The MVVM Toolkit also has a feature currently in preview that makes the code even less verbose, thanks to **source generators**. It's a C# feature introduced in .NET 5 that enables developers to generate code on the fly that gets added during the compilation phase. Thanks to this feature, you can use attributes to decorate your properties, which will generate all the boilerplate code you need to implement the **`INotifyPropertyChanged`** interface. This is how our **`MainViewModel`** can be changed using this feature:

```
public partial class MainViewModel : ObservableObject
{
    [ObservableProperty]
    private string name;
    [ObservableProperty]
    private string surname;
}
```

Neat, isn't it? All we need is to declare our **`MainViewModel`** as a partial class (since the source generator will create, under the hood, another declaration of the class with the code required to support the **`INotifyPropertyChanged`** interface) and decorate two simple private properties with the **`[ObservableProperty]`** attribute. That's it. Now our **`MainViewModel`** class will automatically expose two public properties, called **`Name`** and **`Surname`**, which are able to dispatch notifications to the UI layer.

There's another popular library to implement the MVVM pattern in a XAML-based application—**Prism**. It was born as a project created by a Microsoft division called **Patterns and Practices**; now it's an open source project maintained by two Microsoft MVPs, Brian Lagunas and Dan Siegel, with the support of the community. Unlike the MVVM Toolkit, Prism is a complete framework that, other than giving you the

basic components, also supports many advanced features including the following:

- Creating custom services to support dialogs
- Supporting navigation from a ViewModel
- Enabling the splitting of the UI into multiple components
- Supporting modules, which are a way to split an application into multiple sub-libraries and components that can be dynamically loaded when needed

Prism is more powerful than the MVVM Toolkit, but it has a tougher learning curve. Unlike the MVVM Toolkit, onboarding new developers on a project built with Prism is more challenging, since they must first learn all the key components of the framework.

They're both great choices when it comes to supporting the MVVM pattern in your application: you should evaluate which one works best for you based on your requirements.

NOTE

You can learn more about Prism on the official website at **https://prismlibrary.com/**.

Let's continue our journey of implementing the MVVM by learning another key concept, commands.

Supporting actions with commands

Let's go back to the starting point of our journey into the MVVM pattern: a page where users can fill in their names and surnames. By adding properties and implementing the **`INotifyPropertyChanged`** interface in the ViewModel, we have been able to remove the tight con-

nection between the UI layer and code-behind class when it comes to storing and retrieving the data. But what about the action? Our previous example has a **`Button`** control with an event handler, which includes the code to save the data. However, this is another XAML scenario that creates a deep connection between the code-behind class and the UI layer: an event handler can only be declared in the code-behind. We can't declare it in our ViewModel, where the data we need (the name and surname) is actually stored.

It's time to introduce commands, which are a way to express actions not with an event handler, but through a regular property. This scenario is enabled by a built-in XAML interface called **`ICommand`**, which requires you to implement two properties:

- The first describes the code to execute when the command is invoked.
- Optionally, a condition that must be satisfied for the command to be invoked.

This is another one of the scenarios where the MVVM Toolkit greatly helps. Without it, in fact, you would need to create a dedicated class that implements the **`ICommand`** interface for each action you need to handle in your application. The MVVM Toolkit instead offers a generic class to implement commands, called **`RelayCommand`**. Let's see how we can implement it in our **`MainViewModel`** class:

```
public class MainViewModel : ObservableObject
{
    public MainViewModel()
    {
        SaveCommand = new RelayCommand(SaveAction);
    }
    public RelayCommand SaveCommand { get; set; }
    private void SaveAction()
    {
```

```
            Debug.WriteLine($"{Name} {Surname}");
        }
        private string _name;
        public string Name
        {
            get => _name;
            set => SetProperty(ref _name, value);
        }
        private string _surname;
        public string Surname
        {
            get => _surname;
            set => SetProperty(ref _surname, value);
        }
    }
```

We expose a new **`RelayCommand`** property, called **`SaveCommand`**. In the ViewModel's constructor, we initialize it by passing a method, called **`SaveAction()`**, that we want to execute when the command is invoked. In this example, it just logs the name and surname of the user in the console; in a real-world application, you would use this action to store the user data in a database.

Thanks to the **`RelayCommand`** class, we can expose our action with a simple property instead of an event handler. Now we can connect it to the **`Command`** property exposed by all the XAML controls that handle user interaction, such as the **`Button`** control. Here is how our XAML page gets updated:

```
    <Page>
        <StackPanel>
            <TextBlock Text="Name" />
            <TextBox Text="{x:Bind ViewModel.Name,
              Mode=TwoWay}" />
            <TextBlock Text="Surname" />
            <TextBox Text="{x:Bind ViewModel.Surname,
```

```
            Mode=TwoWay}" />
          <Button Content="Add" Command="{x:Bind
            ViewModel.SaveCommand}"/>
      </StackPanel>
  </Page>
```

As you can see, we can treat the command like any other property exposed by the ViewModel, which means that we can simply use **`x:Bind`** to connect it to the **`Button`** control.

Commands can be implemented using the new preview feature of the MVVM Toolkit based on source generators. This is how the previous code can be simplified:

```
  [ICommand]
  private void Save()
  {
      Debug.WriteLine($"{Name} {Surname}");
  }
```

Instead of manually creating a new **`RelayCommand`** object, we just define the action we want to execute, and we decorate it with the **`[ICommand]`** attribute. The toolkit will automatically generate a command object named like the method followed by the **`Command`** suffix. In the previous example, the name of the generated command will be **`SaveCommand`**, since the method name is **`Save()`**. This feature helps you to make your ViewModel even easier to read.

There's also another implementation of the **`RelayCommand`** class, which supports passing a parameter from the View to the ViewModel via the **`CommandParameter`** property. For example, this is an alternative way to pass the name of the user to the command:

```
  <Button Content="Add" Command="{x:Bind
    ViewModel.SaveCommand}"CommandParameter="{x:Bind
          ViewModel.Name, Mode=OneWay}"/>
```

To support this approach, you must use the **`RelayCommand<T>`** implementation, where **`T`** is the type of data you have set in the **`CommandParameter`** property. In the previous example, since **`Name`** is a string, this is how the ViewModel implementation changes:

```
public MainViewModel()
{
    SaveCommand = new RelayCommand<string>(SaveAction);
}
public RelayCommand<string> SaveCommand { get; set; }
private void SaveAction(string name)
{
    Console.WriteLine(name);
}
```

Thanks to this implementation, the parameter is directly passed as an argument of the **`SaveAction()`** method.

A common scenario in modern application development is to use asynchronous APIs, which are implemented using the **`async`** and **`await`** pattern in C#. If your command must perform one or more asynchronous calls, you can use the **`AsyncRelayCommand`** class (or the **`AsyncRelayCommand<T>`** one in case you want to pass a parameter). Let's take a look at the following example:

```
public MainViewModel()
{
    SaveCommand = new
AsyncRelayCommand(SaveActionAsync);
}
public AsyncRelayCommand SaveCommand { get; set; }
private async Task SaveActionAsync()
{
    await dataService.SaveAsync(Name, Surname);
}
```

`SaveActionAsync()` is an asynchronous method since inside it we're calling an asynchronous API with the **`await`** prefix. As such, it's masked with the **`async`** keyword, and it returns **`Task`** instead of **`void`**. Being asynchronous, we must wrap it using an **`AsyncRelayCommand`** object, which will enable us to perform the operation on a background thread, leaving the UI thread free.

However, commands aren't just a way to expose actions through properties. They give you more powerful options compared to a normal event handler. Let's take a look.

Enabling or disabling a command

As I mentioned at the beginning of this section, the **`ICommand`** interface also supports defining a condition to enable or disable the command. This feature is very powerful because it's automatically reflected on the visual layer of the control, which is connected to the command via binding. For example, if a command is connected to a **`Button`** control and the command is disabled, the **`Button`** control will also be disabled, blocking the user from clicking it.

Let's change our code so that the **`Button`** control to save a person to the database is enabled only if the **`Name`** and **`Surname`** properties actually contain values:

```
public class MainViewModel : ObservableObject
{
    public MainViewModel()
    {
        SaveCommand = new RelayCommand(SaveAction,
            () => !string.IsNullOrWhiteSpace(Name) &&
              !string.IsNullOrWhiteSpace(Surname));
    }
    public RelayCommand SaveCommand { get; set; }
    private void SaveAction()
```

```
    {
        Console.WriteLine($"{Name} {Surname}");
    }
}
```

Now we are passing a second parameter when we initialize the **SaveCommand** object: a **boolean** function, which must return **true** when the command should be enabled, and **false** when it should be disabled instead. In our scenario, this condition is based on the values of the **Name** and **Surname** properties: if one of them is empty, we disable the button, since we don't have all the information we need to save the user in the database.

However, this code isn't enough. If you try the application, you will notice that the **Button** control will indeed be disabled at startup, but it won't be enabled again once you have filled the two **TextBox** controls with some values. The reason is that, by default, the **boolean** condition we have set in the **SaveCommand** object is evaluated only the first time the command gets initialized. We must evaluate the condition, instead, every time the value of the **Name** or **Surname** properties changes, since they are the two ones that can influence the status of the command. To support this requirement, the **RelayCommand** class exposes a method called **NotifyCanExecuteChanged()** that we must call every time one of our properties changes. This is how our **ViewModel** looks after we make this change:

```
public class MainViewModel : ObservableObject
{
    public MainViewModel()
    {
        SaveCommand = new RelayCommand(SaveAction,
            () => !string.IsNullOrWhiteSpace(Name) &&
              !string.IsNullOrWhiteSpace(Surname));
    }
    public RelayCommand SaveCommand { get; set; }
```

```
        private void SaveAction()
        {
            Console.WriteLine($"{Name} {Surname}");
        }
        private string _name;
        public string Name
        {
            get => _name;
            set
            {
                SetProperty(ref _name, value);
                SaveCommand.NotifyCanExecuteChanged();
            }
        }
        private string _surname;
        public string Surname
        {
            get => _surname;
            set
            {
                SetProperty(ref _surname, value);
                SaveCommand.NotifyCanExecuteChanged();
            }
        }
    }
```

The **Name** and **Surname** properties, other than calling the **SetProperty()** method in the setter, also invoke the **NotifyCanExecuteChanged()** method exposed by the **SaveCommand** object. Thanks to this change, our application will now behave as expected. As soon as you fill in some values both in the **Name** and **Surname** fields, the **Button** control will be enabled, allowing the user to complete the action.

Now that we have learned the basic concepts of the MVVM pattern (binding and commands), let's see how we can make it even more ef-

fective by adopting a pattern called dependency injection.

Making your application easier to evolve

When you build applications with the MVVM pattern, one of the consequences is that you start to move all the data layers into separate classes, maybe even into separate class libraries. As mentioned at the beginning of this chapter, in fact, ideally the model should be completely platform-agnostic and you should be able to reuse the same classes also in any other .NET project, such as a Blazor web app or an ASP.NET Web API.

Let's come back to our example of a page that contains a form to add a new user. If we follow the principle of isolating the model, it means that your ViewModel should never contain code like this:

```
public MainViewModel()
{
    SaveCommand = new RelayCommand(SaveAction,
        () => !string.IsNullOrEmpty(Name) &&
          string.IsNullOrEmpty(Surname));
}
public RelayCommand SaveCommand { get; set; }
private void SaveAction()
{
    using (var context = new UserContext())
    {
        var person = new Person()
        {
            Name = Name,
            Surname = Surname
        };
        context.People.Add(person);
```

```
            context.SaveChanges();
        }
    }
```

This implementation of the **`SaveAction()`** method (even if simplified, since it lacks many other elements) is using the Entity Framework Core library to add a new user to the database. However, the ViewModel isn't the right place to contain such data logic.

Instead, you should have a similar implementation, where the data logic is wrapped in a dedicated class (**`DatabaseService`** in the following example), which is used by the ViewModel:

```
    private void SaveAction()
    {
        DatabaseService dbService = new DatabaseService();
        Person person = new Person
        {
            Name = Name,
            Surname = Surname
        };
        db.SavePerson(person);
    }
```

This means that the ViewModel now has a dependency from the **`DatabaseService`** class since, without it, it wouldn't be able to work properly. In the previous example, we have created a new instance of the **`DatabaseService`** class directly in the ViewModel which, however, can lead to many problems as the application grows:

- One of the best practices to adopt in software development when it comes to building complex projects is **unit testing**. This book isn't the right place to discuss such a deep topic in detail, so you can refer to the official Microsoft documentation at **https://docs.microsoft.com/en-us/visualstudio/test/unit-test-basics** as a starting point. The goal of unit testing is to help you evolve

your application more safely, by making sure that when you add new code or make changes you don't break existing features.

A unit test should be as small as possible and must focus on testing a specific logic. However, the previous code isn't a good fit for a unit test, because we are using a **`DatabaseService`** object, which performs operations on a real database. Let's say now that our unit test of the **`SaveAction()`** method fails. Did it happen because the logic is incorrect? Or because there was a problem with the connection to the database? To properly execute the unit tests, you must abstract the **`DatabaseService`** class, but the current implementation makes it impossible.

- The application becomes hard to evolve and adapt to requirement changes. Let's assume that, at some point, your application needs to scale more efficiently, so you plan to switch your data layer from a relational database based on SQL Server to a document database based on Azure Cosmos DB. You would need to go into each ViewModel of your application and manually replace the initialization of the data layer with another service to replace the **`DatabaseService`** class.

To solve these challenges, the MVVM pattern is often paired with another pattern called **Inversion of Control**. When you adopt this approach, you change the perspective around object initialization. Instead of you as the developer manually creating new instances of a class, it's the application itself that takes care of initializing the objects whenever it's needed.

The basic building block to implement this pattern is a container, which you can think of like a box inside which you can store all the classes you're going to need in your application. Whenever any actor in your application needs one of these classes, you won't manually create a new instance, but you will ask the container to give it one for you.

Let's see step by step how to implement this approach. The first requirement is to create an interface to describe our service. An interface describes only the properties or methods exposed by a class, without defining the implementation. This is how the interface of our **`DatabaseService`** class would look:

```
public interface IDatabaseService
{
    public void SavePerson(Person person);
}
```

By providing an interface, we make it easier to address the two challenges we have previously outlined:

- If I need to write unit tests for a ViewModel, I can provide a fake implementation (called a **mock**) of the **`DatabaseService`** that doesn't require a real connection with the database. There are many mocking libraries in the .NET ecosystem (the most popular one is **`Moq`**) that enable you to create a fake implementation of an object and return fake data whenever a method is called.
- If you need to swap the **`DatabaseService`** implementation to switch from SQL Server to Azure Cosmos DB, as long as your new class implements the same **`IDatabaseService`** interface, you don't need to change the ViewModel implementation.

Now we need a **`DatabaseService`** class that implements the **`IDatabaseService`** interface. In our case, it will contain our logic to add a new item to the database (the following snippet is a simplification since it lacks all the code required to initialize Entity Framework Core):

```
public class DatabaseService : IDatabaseService
{
    public void SavePerson(Person person)
    {
        using (var context = new UserContext())
```

```
        {
            context.People.Add(person);
            context.SaveChanges();
        }
    }
}
```

Now we have everything we need to effectively manage the **`DatabaseService`** dependency. However, we are still missing something important—the container. There are many libraries that give you a container ready to be used: some of the most popular ones are Ninject, Unity, and Castle Winsdor. In this book, we're going to use the Microsoft implementation provided for the .NET ecosystem, which is available through the NuGet package called **`Microsoft.Extensions.DependencyInjection`**. This is a perfect companion for the MVVM Toolkit since it integrates with the Inversion-of-Control support it offers through a series of classes included in the **`CommunityToolkit.Mvvm.DependencyInjection`** namespace.

Thanks to these libraries, we can use the **`OnLaunched`** event of the **`App`** class to set up our container and store inside it all the building blocks we need for our application:

```
protected override void OnLaunched(Microsoft.UI.Xaml
  .LaunchActivatedEventArgs args)
{
    m_window = new MainWindow();
    Ioc.Default.ConfigureServices(new
ServiceCollection()
        .AddSingleton<IDatabaseService,
DatabaseService>()
        .BuildServiceProvider()
    );
    m_window.Activate();
}
```

The container is exposed as a singleton (so a static instance is shared across all the applications) through the **`Default`** property of the **`IoC`** class. We use the **`ConfigureServices()`** method to set up the container by passing a **`ServiceCollection()`** object that registers all the dependencies we need to use in our application. In this example, we are creating a connection between the **`IDatabaseService`** interface and the **`DatabaseService`** class. Whenever we need an **`IDatabaseService`** implementation in our ViewModels, the container will return to us a concrete **`DatabaseService`** object. By referencing only interfaces in our ViewModel instead of concrete classes, we make sure that we can easily swap the implementation if we need. The connection is created with the **`AddSingleton()`** method, which means that every ViewModel will receive the same instance of the **`DatabaseService`** class.

Now we have a container that contains everything we need to bootstrap our application. How do we retrieve the classes that we need? One way could be to use the **`Ioc`** singleton in the **`SavePerson()`** method of our ViewModel to retrieve the **`DatabaseService`**, instead of manually creating a new instance, as in the following example:

```
private void SaveAction()
{
    IDatabaseService dbService =
      Ioc.Default.GetService<IDatabaseService>();
    Person person = new Person
    {
        Name = Name,
        Surname = Surname
    };
    dbService.SavePerson(person);
}
```

However, this approach isn't the best one, since we still need to manually retrieve a reference to each of the services we need to use in our ViewModel.

The best approach is to adopt a pattern called **dependency injection**, which means creating a dependency between the ViewModel and the services that it needs to work properly. Then, instead of asking the container for an instance of each service, we directly ask for an instance of the ViewModel. If all the dependencies are properly registered, the container will give us a ViewModel instance with all the services we need already injected.

But how do we create such a tight dependency? It's easy in C#: we just need to add our services as constructor parameters of the ViewModel. For example, this is how we create a dependency between the **`MainViewModel`** class and an **`IDatabaseService`** object:

```
public class MainViewModel : ObservableObject
{
    private IDatabaseService _databaseService;
    public MainViewModel(IDatabaseService
databaseService)
    {
        this._databaseService = databaseService;
    }
}
```

After this change, we won't be able to create a new instance of the **`MainViewModel`** class anymore without passing, as a parameter, an object that implements the **`IDatabaseService`** interface. To enable dependency injection, however, we need to make a change to the code in the **`App`** class that initializes the container:

```
protected override void OnLaunched(Microsoft.UI.Xaml
  .LaunchActivatedEventArgs args)
{
    m_window = new MainWindow();
    Ioc.Default.ConfigureServices(new
ServiceCollection()
```

```
            .AddSingleton<IDatabaseService,
    DatabaseService>()
            .AddTransient<MainViewModel>()
            .BuildServiceProvider()
        );
        m_window.Activate();
    }
```

Other than the **`IDatabaseService`** interface, we must register the ViewModel inside the container as well. In this case, it's registered with the **`AddTransient()`** method, which means that we'll get a new instance of the class every time someone requests it. You can see how, in this case, we aren't creating a connection with an interface, which means that the container will simply create a new instance of the **`MainViewModel`** class. In our scenario, it's a valid solution because we are forecasting that at some point, we might need to change the **`DatabaseService`** implementation, while the **`MainViewModel`** will always stay the same. However, for more complex applications, it's good practice to describe ViewModels with an interface as well.

The last step we are missing is to supply the ViewModel to the page from the container instead of manually creating a new instance. We'll have to switch the initializer in the **`MainPage`** class to the following one:

```
public sealed partial class MainPage : Page
{
    public MainViewModel ViewModel { get; set; }
    public MainPage()
    {
        this.InitializeComponent();
        ViewModel = Ioc.Default.GetService
          <MainViewModel>();
    }
}
```

We aren't manually creating a new instance of the **`MainViewModel`** class anymore for the **`ViewModel`** property (which we use to connect the controls in the UI through binding) – instead, we're getting it from our container. Since the **`MainViewModel`** class has a dependency from the **`IDatabaseService`** interface, it means that the container will check whether we have registered any class that implements it. In our case, the answer will be affirmative, so our **`MainPage`** will be instantiated without errors. If by any chance, we had forgotten to register the **`DatabaseService`** class in the container, we would have received an error since the container doesn't know how to resolve the dependency.

Thanks to Inversion of Control and dependency injection, we have solved all the challenges that we outlined at the beginning of the section:

- Since the **`MainViewModel`** class has a dependency from an interface (**`IDatabaseService`**) instead of a concrete class (**`DatabaseService`**), it means that, when we write unit tests, we can simply provide a fake implementation of the interface that, instead of working with the real database, returns mock data. This is an example of this approach:

```
[TestMethod]
public void SaveCommand_CannotExecute
  _WithInvalidProperties()
{
    // Arrange.
    var dataServiceMock = new
      Mock<IDatabaseService>();
    var vm = new MainViewModel
      (dataServiceMock.Object);
    //Act
    vm.Name = string.Empty;
    vm.Surname = string.Empty;
    // Assert.
    Assert.IsFalse(vm.SaveCommand.CanExecute(null));
```

```
}
```

This unit test verifies that the **SaveCommand** command can't be executed if the **Name** and **Surname** properties of the ViewModel are empty. Thanks to the usage of dependency injection, we can use the **Moq** library to create a mock implementation of the **IDatabaseService** interface. This way, we can test whether the **MainViewModel** behavior is correct without having to establish a connection with a real database.

- If we need to switch our implementation from a classic SQL Server database to Azure Cosmos DB, we just need to create a new class that implements the **IDatabaseService** and, in the **App** class, register it in the container as a replacement of the existing one. For example, let's say we have created a new class called **CosmosDbService** with the following implementation:

```
public class CosmosDbService : IDatabaseService
{
    public void SavePerson(Person person)
    {
        this.cosmosClient = new
          CosmosClient(EndpointUri, PrimaryKey);
        this.database = await this.cosmosClient
          .CreateDatabaseIfNotExistsAsync();
        this.container = await this.database
          .CreateContainerIfNotExistsAsync
            (containerId, "/Surname");
        await this.container.CreateItemAsync
          <Family>(person, new PartitionKey
            (person.Surname));
    }
}
```

This implementation is simplified of course, but it's using the .NET SDK for Azure Cosmos DB to implement the **SavePerson()** method. Even if the implementation is different, the signature is still the same,

which means that we don't have to change anything in the ViewModel to use it.

We just need to go to the **App** class and change the initialization code of the container to be like the following sample:

```
protected override void OnLaunched(Microsoft.UI.Xaml
  .LaunchActivatedEventArgs args)
{
    m_window = new MainWindow();
    Ioc.Default.ConfigureServices(new
ServiceCollection()
        .AddSingleton<IDatabaseService,
CosmosDbService>()
        .AddTransient<MainViewModel>()
        .BuildServiceProvider()
    );
    m_window.Activate();
}
```

Instead of registering the **DatabaseService** class for the **IDatabaseService** interface, we replace it with the **CosmosDbService** class. Now, regardless of the number of ViewModels that our application has, if any of them have a dependency on the **IdatabaseService** interface, they will start to use the **CosmosDbService** class right away.

Dependency injection can help to solve many problems in real-world application development, but it doesn't solve one of the challenges you will face when you start to adopt the MVVM pattern: how to exchange data between different classes. Let's learn more in the next section!

Exchanging messages between different classes

Splitting the application into different domains independent of each other provides many great advantages, but it also creates a few challenges. For example, there are scenarios where you need to exchange data between two ViewModels; or you need the ViewModel to communicate with the View to trigger an operation on the UI layer, such as starting an animation.

Creating a dependency between multiple elements would break the MVVM pattern, so we must find another solution. One of the most adopted solutions to support these scenarios is the **Publisher-Subscriber pattern**, which enables every class in the application to exchange messages. What makes this pattern a great fit for applications built with the MVVM patterns is that these messages aren't tightly coupled: a publisher can send a message without knowing who the subscribers are, and subscribers can receive messages without knowing who the publisher is.

Let's see a concrete example in the following scenario, based on the previous example: when the `SaveCommand` is executed, we want to trigger an animation defined on the page with a `Storyboard`. If you remember what we have learned in ***Chapter 2***, *The Windows App SDK for a Windows Forms Developer*, animations can be triggered in the code-behind by calling the `Begin()` method exposed by the `Storyboard` class. This is an example of code that, even if you adopt the MVVM pattern, should stay in the code-behind rather than the ViewModel. We're talking, in fact, about code that deeply interacts with the UI layer. However, due to the separation of concerns, we have a disconnection: the `SaveCommand` task is handled in the ViewModel, while the animation must be triggered in the code-behind. We're going to use messages to handle the communication between these two layers, without creating a tight dependency.

Let's start by creating our animation in XAML. We're going to hide the button once the user has clicked on it:

```
<Page>
    <Page.Resources>
        <Storyboard x:Key="AddButtonAnimation">
            <DoubleAnimation
              Storyboard.TargetName="AddButton"
              Storyboard.TargetProperty="Opacity"
              From="1.0" To="0.0" Duration="0:0:3"
              />
        </Storyboard>
    </Page.Resources>
    <StackPanel>
        <TextBlock Text="Name" />
        <TextBox Text="{x:Bind ViewModel.Name,
          Mode=TwoWay}" />
        <TextBlock Text="Surname" />
        <TextBox Text="{x:Bind ViewModel.Surname,
          Mode=TwoWay}" />
        <Button x:Name="AddButton" Content="Add"
          Command="{x:Bind ViewModel.SaveCommand}" />
    </StackPanel>
</Page>
```

Now we need a message that the ViewModel will send to the View when the animation must be triggered. A message is nothing more than a C# class: it can be a plain one (if the message should act just as a notification that an event has occurred) or it can contain one or more properties if you need to also exchange some data. Our use case is the first scenario (we must notify the UI layer when the animation should start), so this is how our message looks:

```
public class StartAnimationMessage
{
    public StartAnimationMessage() { }
}
```

Now we are ready to dispatch this message. We do it inside the **`SaveAction()`** method of our **`MainViewModel`** class, which is invoked

when the user presses the **Save** button through the **SaveCommand** object:

```
public void SaveAction()
{
            _databaseService.SavePerson(new Person {
Name =
              Name, Surname = Surname });
            WeakReferenceMessenger.Default.Send(new
              StartAnimationMessage());
}
```

WeakReferenceMessenger is a class offered by the MVVM Toolkit and is included in the **CommunityToolkit.Mvvm.Messaging** namespace. As the name says, it uses a weak approach to register messages, which means that unused recipients are still eligible for garbage collection, reducing the chances of memory leaks. The class exposes a singleton through the **Default** object since we need a central point of registration for messages if we want all the classes to be able to exchange them. To send the message, we simply call the **Send()** method, passing as a parameter a new instance of our **StartAnimationMessage** class. As you can see, we don't have any reference to the recipient. Our ViewModel continues to be independent of the View.

Let's move now to the View – let's subscribe to receive this message in the code-behind class of **MainPage**:

```
public partial class MainPage : Page,
  IRecipient<StartAnimationMessage>
{
    public MainPage()
    {
        this.InitializeComponent();
        WeakReferenceMessenger.Default.
          Register<StartAnimationMessage>(this);
    }
```

```
    public void Receive(StartAnimationMessage message)
    {
        Storyboard sb = this.Resources
          ["AddButtonAnimation"] as Storyboard;
        sb.Begin();
    }
}
```

As a first step, in the page constructor, we again use the **`WeakReferenceMessenger`** class and its singleton, this time to register for incoming messages using the **`Register<T>()`** method, where **`T`** is the type of message want to receive (in our case, it's **`StartAnimationMessage`**). As a parameter, we must pass the class that will handle the message: we use the **`this`** keyword, since we want the code-behind class itself to manage it.

As a second step, we let the page implement the **`IRecipient<T>`** interface, where **`T`** is the type of message we're subscribing to. By adding this interface, we are required to implement the **`Receive()`** method, which is triggered when there's an incoming message. As a parameter, we get a reference to the message itself, so we can retrieve data from its properties if needed. In our scenario, we're using the message just as a notification, so all we do is retrieve a reference to the **`Storyboard`** object we have created in XAML and invoke the **`Begin()`** method to run the animation. Also, in this case, note how we don't have a direct reference to the **`MainViewModel`** class that sent the message.

If you prefer not to implement an interface, there's also another way to subscribe for messages:

```
public partial class MainPage : Page
{
    public MainPage()
    {
        InitializeComponent();
        WeakReferenceMessenger.Default.Register
```

```
            <StartAnimationMessage>(this,
              (sender, message) =>
          {
              Storyboard sb = this.Resources
                ["AddButtonAnimation"] as Storyboard;
              sb.Begin();
          });
      }
  }
```

When you call the **`Register()`** method, you can pass as a second parameter a function with the code you want to execute when the message is received.

Thanks to the Publisher-Subscriber pattern, we have achieved our goal: when the user clicks on the **Save** button on the page, it will disappear after a few seconds thanks to the animation. Most of all, we have implemented this without creating a tight relationship between the View and ViewModel, which are still decoupled.

Let's see now that last topic of our journey of building a future-proof architecture with the MVVM pattern: managing the navigation.

Managing navigation with the MVVM pattern

We explored navigation in ***Chapter 5***, *Designing Your Application*, and we have learned that it's a critical component of a Windows application. Especially if you're building enterprise applications, it's very likely that you will have multiple pages and the various interactions will lead the user to move from one to another. However, it's also one of the most challenging features to implement when you start adopting the MVVM pattern. The reason is that the goal of the pattern is to decouple the UI from the business logic; however, this creates a fracture in this case. The actions performed by the user are managed by

the ViewModel, while navigation is managed in the View layer by using the **`Frame`** class to trigger the navigation and by subscribing to events such as **`OnNavigatedTo()`** and **`OnNavigateFrom()`** to manage the life cycle of a page.

There are multiple solutions to this challenge, but the most common one is based on the implementation of a class called **`NavigationService`**, which can wrap the access to the underline **`Frame`** of the application so that you can trigger the navigation from one page to another directly to a ViewModel. Many advanced MVVM frameworks, such as Prism, provide an implementation of this class that, by using dependency injection, you can inject into your ViewModels.

The MVVM Toolkit doesn't provide it, so we're going to reuse the implementation included in Windows Template Studio, which is available at **https://github.com/microsoft/`WindowsTemplateStudio`**. It's a Visual Studio extension that you can use to accelerate the bootstrap of a new Windows application by giving you many building blocks that makes it easier to create a complex architecture by supporting the MVVM pattern and by providing services to enable navigation, storage access, and much more. At the time of writing, Windows Template Studio doesn't support Visual Studio 2022 and Windows App SDK 1.0 yet and as such, we won't cover it in this book. However, we're going to reuse the same **`NavigationService`** implementation so that we can adopt it in our application.

Specifically, we're going to reuse the **`NavigationService`** class and the **`INavigationService`** and **`INavigationAware`** interfaces from the WinUI project created by the template.

In this section, we won't focus on how **`NavigationService`** is implemented or how it works under the hood. You can take a look at the implementation in the example application at **https://github.com/PacktPublishing/Modernizing-Your-Windows-**

Applications-with-the-Windows-Apps-SDK-and-WinUI/tree/main/Chapter06/05-Navigation.

We'll focus, instead, on how we can use it to handle the navigation. Let's start!

Navigating to another page

The first step is to create a second page in our application with a dedicated ViewModel.In our scenario, we have called it **`DetailPage`**, which is backed by the **`DetailPageViewModel`** class.

Now we must register **`NavigationService`** in our container so that we can inject it into our ViewModels. The **`NavigationService`** comes with an interface that describes it called **`INavigationService`**, so we can just add it to our container using the approach we learned in this chapter. We also register the new **`DetailPageViewModel`** we have just created. We achieve this goal by adding the following code in the **`OnLaunched`** event of the **`App`** class:

```
Ioc.Default.ConfigureServices(new ServiceCollection()
    .AddSingleton<INavigationService,
NavigationService>()
    .AddSingleton<IDatabaseService, DatabaseService>()
    .AddTransient<MainPageViewModel>()
    .AddTransient<DetailPageViewModel>()
    .BuildServiceProvider());
```

The **`NavigationService`** is registered as a singleton so that the same instance can be reused across every ViewModel.

The main window of the application is configured in the same way that we learned in ***Chapter 5***, *Designing Your Application*. The core is a **`Frame`** control, which we're going to use to load the pages and handle the navigation. This is a basic implementation:

```
<Window>
    <Frame x:Name="ShellFrame" />
</Window>
```

In the code-behind, we use the **`Frame`** reference to load the default page of the application, the one called **`MainPage`**:

```
public sealed partial class MainWindow : Window
{
    public MainWindow()
    {
        this.InitializeComponent();
        ShellFrame.Content = new MainPage();
    }
}
```

Now we are ready to start using the **`NavigationService`** class in our ViewModel. If you remember the previous implementation we saw of the **`MainPageViewModel`** class, we have a **`RelayCommand`** called **`SaveCommand`**, which triggers a method called **`SaveAction()`**. Let's change it to trigger navigation to the **`DetailPage`** application we previously created:

```
public class MainPageViewModel : ObservableObject
{
    private readonly INavigationService
_navigationService;
    public MainPageViewModel(INavigationService
      navigationService)
    {
        SaveCommand = new RelayCommand(SaveAction,
            () => !string.IsNullOrWhiteSpace(Name) &&
              !string.IsNullOrWhiteSpace(Surname));
        _navigationService = navigationService;
    }
    public RelayCommand SaveCommand { get; set; }
    private void SaveAction()
```

```
        {
            _navigationService.NavigateTo(typeof(DetailPage
    ));
        }
    }
```

We simply need to add an **`INavigationService`** parameter to the constructor of the ViewModel. Thanks to dependency injection, an instance of the **`NavigationService`** class will be automatically injected inside the ViewModel. Now we can trigger the navigation by calling the **`NavigateTo()`** method exposed by the service, passing the page type as a parameter.

NOTE

If you want to implement a cleaner approach, you can change the **`NavigationService`** *implementation to use a key (such as the page's name) instead of the page's type as a parameter of the* **`NavigateTo()`** *method. The page's type, in fact, creates a relationship between the ViewModel and the View, so it might create challenges when you want to reuse the same ViewModel in other types of projects.*

The command will now redirect the user to the **`DetailPage`** of the application. However, what if we want to pass some data to it? Let's explore a solution.

Passing parameters from a ViewModel to another

To handle this scenario, the **`NavigationService`** automatically subscribes to the navigation event exposed by the **`Page`** class, such as **`OnNavigatedTo()`** and **`OnNavigatedFrom()`**. We can easily access them through another helper that we have imported from Windows Template Studio: the **`INavigationAware`** interface. By implementing this interface, we'll be able to manage the navigation events directly in the ViewModels.

Let's start by passing a parameter to the navigation action, which works in the same way that we learned in ***Chapter 5***, *Designing Your Application*:

```
private void SaveAction()
{
    Person person = new() { Name = Name, Surname =
Surname };
    _navigationService.NavigateTo(typeof(DetailPage),
      person);
}
```

We have created a new **`Person`** object and we have passed it as the second parameter of the **`NavigateTo()`** method exposed by **`NavigationService`**.

Now we can implement the **`INavigationAware`** interface in our **`DetailPageViewModel`**, which gives us the option to receive the **`Person`** object:

```
public class DetailPageViewModel: ObservableObject,
  INavigationAware
{
    private string _name;
    public string Name
    {
        get { return _name; }
        set { SetProperty(ref _name, value); }
    }
    private string _surname;
    public string Surname
    {
        get { return _surname; }
        set { SetProperty(ref _surname, value); }
    }
    public void OnNavigatedTo(object parameter)
```

```
        {
            if (parameter != null)
            {
                Person person = parameter as Person;
                Name = person.Name;
                Surname = person.Surname;
            }
        }
        public void OnNavigatedFrom()
        {
        }
    }
```

Thanks to this new interface, we can use the **`OnNavigatedTo()`** method in our ViewModel to get notified when the navigation to the page connected to this ViewModel has happened. Thanks to the parameter, we can get access to the **`Person`** object that we previously passed to the **`NavigateTo()`** method of **`NavigationService`**. And also, because of this object, we can fill two properties (**`Name`** and **`Surname`**) whose values are displayed in the View thanks to the binding.

This topic concludes our journey into the MVVM pattern. In the next chapter, we're going to put into practice many of these concepts by migrating a Windows application to Windows App SDK and WinUI.

Summary

One of the biggest challenges in software development is building projects that can survive the test of time. Of course, we can't control every aspect: there are many external factors outside our control, like the advent of new and more powerful technologies. However, by choosing the right architecture, we can protect our application from aging too fast. The MVVM pattern is a great example of how you can define the architecture of your Windows applications in a way that makes them easier to maintain, evolve, and test over time. By decou-

pling the UI from the business logic and the data layer, it becomes easier to create modular applications, in which you can replace one component or another when a new requirement arrives, rather than having to rewrite the application from scratch.

At the same time, however, you must remember that architectural patterns are best considered as guidance to help you build better applications, not a set of rules that you have to follow blindly. You must always find the right balance between good architecture and simplicity, otherwise you risk falling into the opposite problem: over-engineering.

The knowledge we acquired in this chapter will be very valuable in the next one: we're going to migrate a real application to WinUI and the Windows App SDK, and as part of the journey, we're going also to evolve it from the traditional code-behind approach to the MVVM pattern.

Questions

1. Using a dedicated library or framework is a critical requirement to implement the MVVM pattern. True or false?
2. What's the main advantage of using messages and the Publisher-Subscriber pattern in an MVVM architecture?
3. It's fine to implement the **`INotifyPropertyChanged`** interface in classes that are considered part of the Model domain. True or false?